Chapter 11 worked solutions

DS11-01 • B-tree insertion and deletion

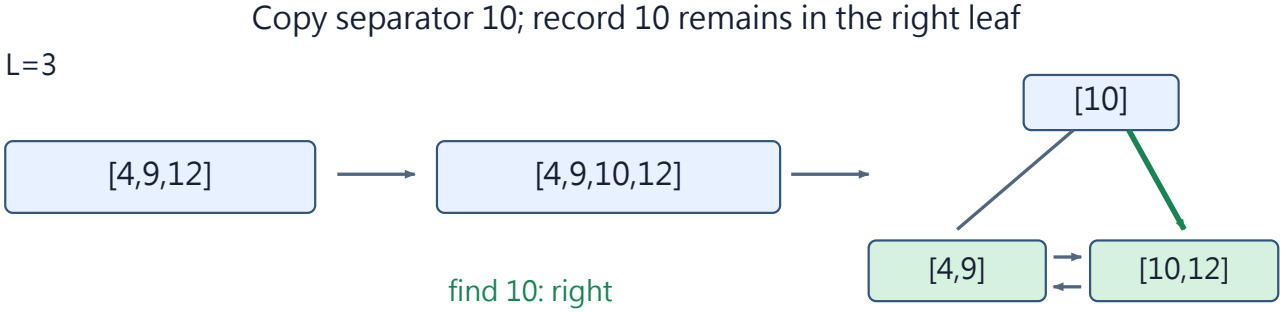
After 10,20,5 the root is `[5,10,20]`. Split before inserting 6, promoting 10 and leaving leaves `[5]`, `[20]`. Insert 6 into `[5,6]`. Insert 12 and 30 into the right leaf, obtaining `[12,20,30]`. Split it before inserting 25 and promote 20. The root is `[10,20]`, with leaves `[5,6]`, `[12]`, `[25,30]`. Before deleting 12, the middle leaf has t−1=1 key while the left sibling has two. Move 6 up to replace parent key 10 and move 10 down to the middle leaf. Delete 12. Final root `[6,20]` has leaves `[5]`, `[10]`, `[25,30]`. Every nonroot leaf contains one through three keys, the root has two keys and three children, and every leaf has depth 1.

DS11-02 • Height and I/O

Depth d≥1 has at least $2t^{d-1}$ nodes, each with at least t−1 keys. Total keys are at least $1 + 2(t-1)(1 + t + \cdots + t^{h-1}) = 2t^h - 1$. With t=64, height 3 needs at least 524,287 keys, whereas height 4 needs at least 33,554,431. The largest permitted integer height is 3. The minimum-capacity inequality alone does not exclude h=2. Its maximum capacity is $(2t)^3 - 1 = 128^3 - 1 = 2{,}097{,}151$, enough for 1,000,000, so height 2 is also feasible. Height counts edges. A root-to-leaf path contains h+1 pages, less a root read when it is cached.

DS11-03 • B+ separators and deletion

The temporary leaf `[4,9,10,12]` splits into `[4,9]` and `[10,12]`. Copy separator 10 into the parent while retaining record 10 in the right leaf. Left.next points right. With a doubly linked chain, right.prev points left. Equality routes right to the record. In the separate L=2 state, deleting 10 leaves `[12]`, satisfying minimum ⌈2/2⌉=1. No borrowing or merge is required, and leaf links stay unchanged. The exact right-minimum convention changes root separator 10 to 12. Leaving it unchanged violates that representation even if some searches still happen to work.



Chapter 11 worked solutions (continued)

DS11-04 • Range-query assumptions

The premise $M \geq 3$ gives $a = \lceil M/2 \rceil \geq 2$, so the logarithm base is valid. For h≥1, the root has at least two children and later internal pages at least a. There are at least $2a^{h-1}$ leaves, so $n \geq 2\ell a^{h-1}$ and $h \leq 1 + \log_a(n/(2\ell))$. Treat h=0 as one page. Location costs O(h+1); except at the two partial ends, every scanned leaf contributes at least ell outputs. Thus index cost is $O(h + 1 + k/\ell)$. With M,L=Θ(B), fixed-size entries/records, and one page per node, this simplifies to $O(1 + \log_B n + k/B)$. Fanout controls location; minimum leaf occupancy controls scanning.

Choose q≥3,M=3,$L = B = 2^q$, two children per internal page, and 2^q full B-record leaves. Then n=B^2 and h=q. Finding the largest key reads q+1 pages, although $\log_B n = 2$. L=Θ(B) alone does not make internal branching grow with B.

Scattered complete records add D data-page reads, worst-case D=k. Clustering in key order at Θ(B) records per data page permits D=O(1+k/B). Producing k outputs independently requires Ω(k) elementary work.

The chapter stores (child_page_id,cached_min) inside one parent page. Changed children return minima along a (page_id,index) ancestor stack; refresh does not fetch all child pages. Internal splitting moves complete cached entries, without persistent child-parent rewrites. Constant page accesses per level give O(h+1) update I/Os. Separate child-resident minima or parent fields can instead add Θ(M) accesses to one refresh/split. Moving entries within a page can still take O(M) CPU work.

DS11-05 • Incompatible order conventions

Minimum degree t is an integer, so t=2.5 is invalid. If A also requires at least ⌈m/2⌉=3 children and at most five, its nonroot nodes contain two through four keys. With t=2, B allows one through three keys. With t=3, it allows two through five. Neither matches. A can split after overflowing to five keys, promote the middle key, and retain two on each side. Our top-down procedure splits a full 2t−1-key node into t−1 keys per side before insertion. Insertion timing, maximum capacity, and split allocation must be specified together.

DS11-06 • B+ underflow, exact separators, and a range

Each nonroot leaf needs ceil(3/2)=2 records. Deleting 10 leaves [12], while both siblings have exactly two records and cannot lend. Merge with [4,9] to obtain [4,9,12], and reconnect [4,9,12]↔[14,16]. The root now has two children and separator [14], the right minimum.

Chapter 11 worked solutions (continued)

Deleting 14 leaves right leaf [16]. The left leaf has three records and lends its largest, 12. Leaves become [4,9]↔[12,16], each with two records; the root separator changes from 14 to 12. Range [9,14] descends left at separator 12, locates and emits 9, follows next, emits 12, then inspects 16 and stops because 16>14. The output is [9,12]; inspecting the stopping key does not output it.

Deleting 4 leaves [9]. The right sibling cannot lend, so merge into [9,12,16]. The internal root has one child and no separator; contract it to the leaf, whose three records fit. Removing routing separator 12 is safe because record 12 already remains in the leaf. Adding that separator as another leaf record would duplicate 12. A B-tree parent key is itself a record and must instead move into the merged child. Discarding it would lose data. The two merge conventions are not interchangeable.

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study